

Manual técnico - ParkNear

Versión: 1.0

Ámbito: Arquitectura, modelos, API REST, integraciones y app móvil (`parknear/`).1.
Visión general

ParkNear es un sistema de **gestión de zonas azules** compuesto por:

- **App móvil** (React Native / Expo) para conductores, encargados y administradores.
- **Microservicios backend** (NestJS) expuestos vía **API Gateway**.
- Integraciones externas: **Mercado Pago** (pagos) y **Google Maps** (mapa en cliente).

Actor	Rol JWT	App
Conductor	<code>CONDUCTOR</code>	Mapa, reserva, pago
Encargado	<code>CONTROLADOR</code>	Gestión de reservas en zona
Administrador	<code>ADMIN</code>	Usuarios, zonas, solicitudes

Gateway de integración: `https://bus.parknear.online`

2. Arquitectura del sistema

2.1 Estilo arquitectónico

Arquitectura **orientada a servicios** con **API Gateway** como único punto de entrada:

Decisión	Detalle
Comunicación	HTTP/REST + JSON
Autenticación	JWT (<code>Authorization: Bearer</code>)
Cliente	No accede a microservicios directamente
BD	Cada servicio NestJS accede a su dominio de datos (Drizzle/ORM)

Decisión	Detalle
Mensajería	No hay broker; flujos síncronos

2.2 Diagrama general

```

flowchart TB
    subgraph clientes [Cliente]
        APP[App móvil Expo]
    end

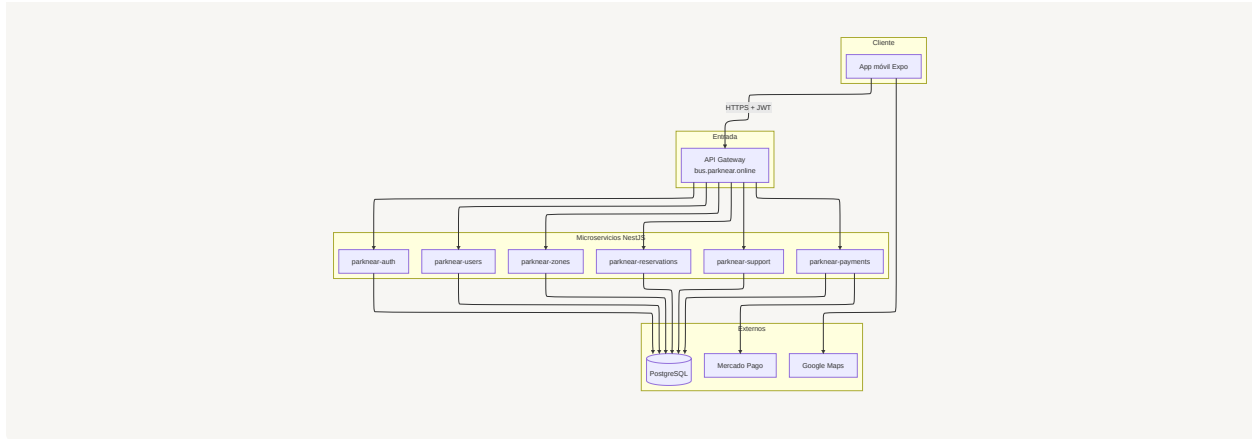
    subgraph gateway [Entrada]
        BUS[API Gateway<br/>bus.parknear.online]
    end

    subgraph servicios [Microservicios NestJS]
        AUTH[parknear-auth]
        USERS[parknear-users]
        ZONES[parknear-zones]
        RES[parknear-reservations]
        PAY[parknear-payments]
        SUP[parknear-support]
    end

    subgraph externos [Externos]
        MP[Mercado Pago]
        GM[Google Maps]
        DB[(PostgreSQL)]
    end

    APP -->|HTTPS + JWT| BUS
    APP --> GM
    BUS --> AUTH & USERS & ZONES & RES & PAY & SUP
    PAY --> MP
    AUTH & USERS & ZONES & RES & PAY & SUP --> DB

```



2.3 Servicios

Servicio	Prefijo API	Responsabilidad
Auth	<code>/auth</code>	Registro, login, JWT
Users	<code>/users</code>	Usuarios, vehículos
Zones	<code>/zonas</code>	Zonas azules, cupos
Reservations	<code>/reservas</code>	Reservas, estados, extensión
Payments	<code>/pagos</code> , <code>/mercadopago</code>	Cobros, webhook MP
Support	<code>/solicitudes</code>	Quejas, solicitudes, evaluaciones

2.4 Capas internas (NestJS)

Cada microservicio sigue el patrón:

```

flowchart LR
    C[Controller] --> S[Service]
    S --> R["Repository / Drizzle"]
    R --> DB["DB (BD)"]
  
```



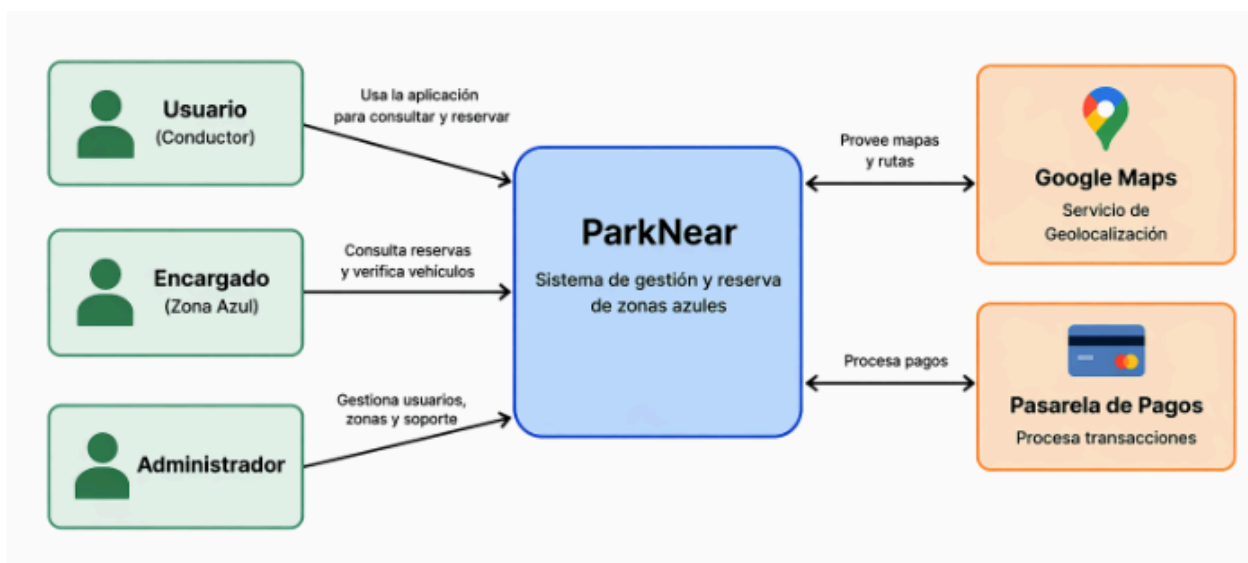
Capa	Función
Controller	Endpoints REST, validación DTO, guards JWT/roles

Capa	Función
Service	Reglas de negocio (precio, estados, cupos)
Repository	Persistencia SQL

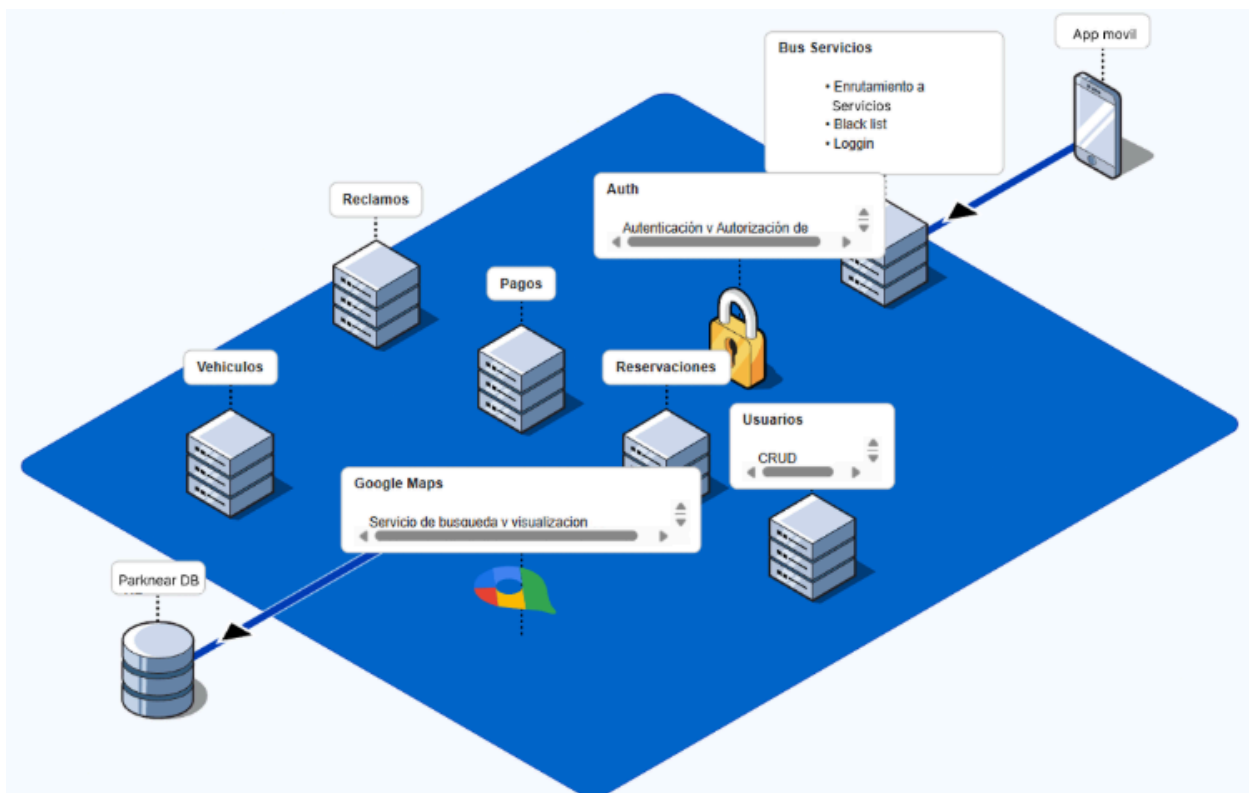
3. Modelo C4

3.1 Nivel 1 — Contexto (C1)

Actores y sistemas externos que interactúan con ParkNear



3.2 Nivel 2 — Contenedores (C2)



3.3 Nivel 3 — Componentes (C3)

Ejemplo: **Reservations Service**

Componente	Archivo	Responsabilidad
ReservasController	reservas.controller.ts	POST/GET/PUT/PATCH /reservas
ReservasService	reservas.service.ts	Precio, cupos, transiciones de estado
ReservasRepository	reservas.repository.ts	Queries a BD

Ejemplo: **Payments Service**

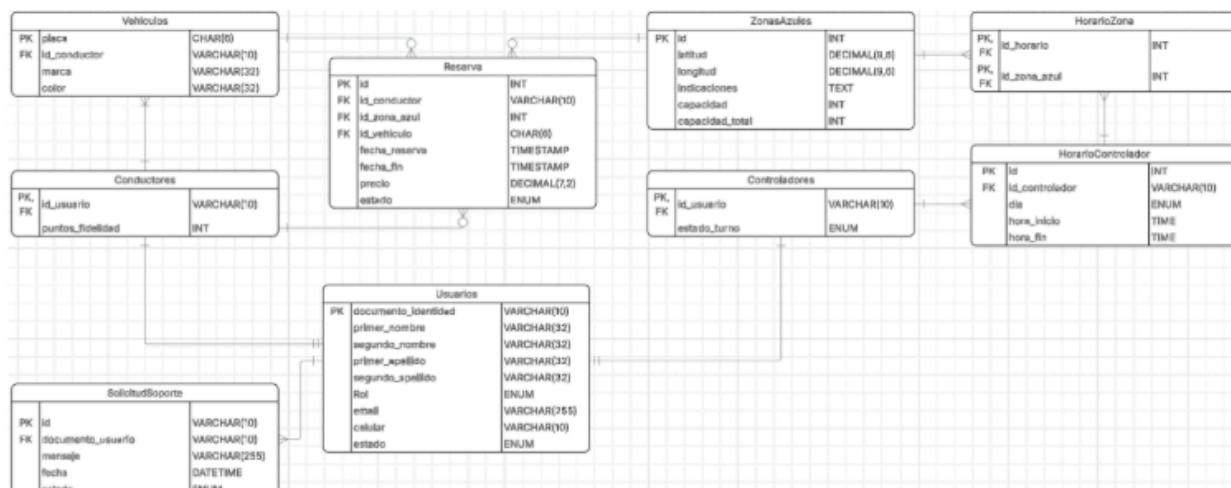
Componente	Responsabilidad
PagosController	POST /pagos , consultas
PagosService	Registro de pago, saldo pendiente
MercadopagoService	Token → cobro MP, webhook
MercadopagoController	/mercadopago/procesar-pago , /notificacion

3.4 Nivel 4 — Código (C4)

Implementación en repositorios GitHub y monorepo local ([parknear-*](#) , [parknear/](#)).
Cada servicio incluye [src/](#) , DTOs, guards JWT, tests y configuración de BD.

4. Modelo de datos

4.1 Diagrama entidad-relación



4.2 Enumeraciones ([parknear/models/enums.ts](#))

Tipo	Valores
EstadoUsuario	activo , no_verificado , inactivo , eliminado
EstadoReserva	pendiente , activa , cancelada , completada
EstadoPago	pendiente , aprobado , rechazado , reembolzado
MetodoPago	mercadopago , efectivo
TipoSolicitud	peticion , queja , reclamo , sugerencia
EstadoSolicitud	pendiente , activa , resuelta

4.3 Ciclo de vida de una reserva

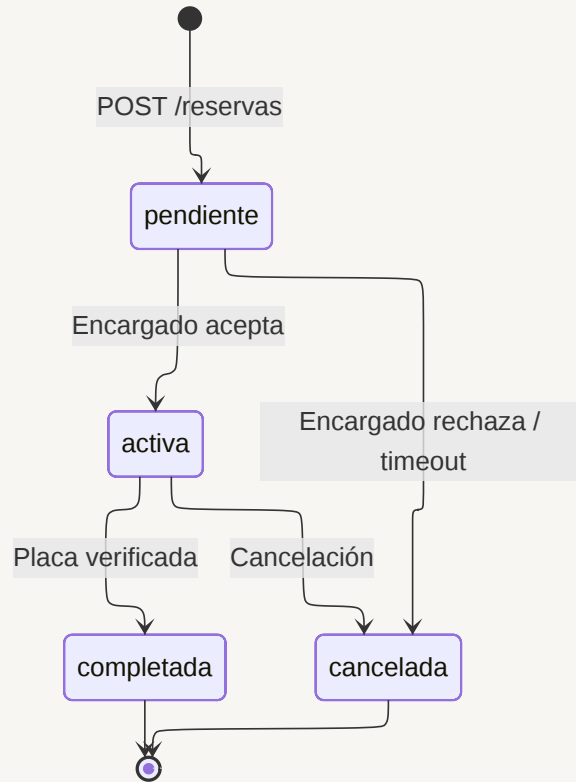
stateDiagram-v2

[*] --> pendiente: POST /reservas

pendiente --> activa: Encargado acepta

pendiente --> cancelada: Encargado rechaza / timeout

activa --> completada: Placa verificada
 activa --> cancelada: Cancelación
 completada --> [*]
 cancelada --> [*]



4.4 Entidad Reserva (respuesta API)

Campo	Tipo	Descripción
<code>id</code>	number	Identificador
<code>id_conductor</code>	UUID	Conductor
<code>id_zona</code>	number	Zona
<code>id_vehiculo</code>	string	Placa (máx. 10)
<code>fecha_real_inicio</code>	ISO datetime	Inicio
<code>fecha_fin</code>	ISO datetime	Fin de ventana
<code>precio</code>	number	COP (calculado en servidor)
<code>estado</code>	EstadoReserva	Estado actual

5. API Gateway, seguridad y roles

5.1 URL base

Variable	Valor
EXPO_PUBLIC_API_GATEWAY_URL	https://bus.parknear.online

Cliente HTTP: `parknear/lib/api-fetch.ts` → `resolveApiUrl(path)` + header Bearer.

5.2 JWT

Respuesta de `POST /auth/login` y `POST /auth/register`:

```
{
  "access_token": "<JWT>",
  "user": {
    "id": "uuid",
    "email": "...",
    "tipo_usuario": "conductor | controlador | admin",
    "documento_identidad": "..."
  }
}
```

Claims relevantes: `sub`, `tipo_usuario`, `email`, `exp`.

Guards en backend: `JwtAuthGuard` + `RolesGuard` (`ADMIN`, `CONDUCTOR`, `CONTROLADOR`).

5.3 Mapeo roles backend ↔ app

<code>tipo_usuario</code>	Guard	App <code>UserRole</code>
<code>conductor</code>	<code>CONDUCTOR</code>	<code>conductor</code>
<code>controlador</code>	<code>CONTROLADOR</code>	<code>encargado</code>
<code>admin</code>	<code>ADMIN</code>	<code>admin</code>

6. Servicios y endpoints REST

Convenciones: rutas relativas al gateway; JSON; errores HTTP estándar (400, 401, 403, 404, 409, 500).

6.1 Parknear Auth Service (`parknear-auth`)

Prefijo: `/auth`

Método	Ruta	Auth	Descripción
POST	<code>/auth/register</code>	No	Registro
POST	<code>/auth/login</code>	No	Inicio de sesión

POST `/auth/register` — Body `RegisterDto`

Campo	Tipo	Requerido
<code>documento_identidad</code>	string	Sí
<code>primer_nombre</code>	string	Sí
<code>segundo_nombre</code>	string	No
<code>primer_apellido</code>	string	Sí
<code>segundo_apellido</code>	string	No
<code>email</code>	string	Sí
<code>contrasena</code>	string	Sí
<code>celular</code>	string	Sí (máx. 13)
<code>estado</code>	EstadoUsuario	No (default <code>inactivo</code>)
<code>tipo_usuario</code>	enum	No (<code>conductor</code> <code>controlador</code> <code>admin</code>)

Respuesta: `{ access_token, user }`.

POST `/auth/login` — Body `LoginDto`

Campo	Tipo	Requerido
<code>email</code>	string	Sí
<code>contrasena</code>	string	Sí

6.2 Parknear Users Service (`parknear-users`)

Prefijo: `/users`

Método	Ruta	Roles
POST	<code>/users</code>	—
GET	<code>/users</code>	ADMIN
GET	<code>/users/:email</code>	ADMIN, CONTROLADOR
POST	<code>/users/:documento/vehículo</code>	ADMIN, CONDUCTOR
PATCH	<code>/users/:documento</code>	ADMIN, CONDUCTOR

POST `/users/:documento/vehículo`

Campo	Tipo	Requerido
<code>placa</code>	string	Sí (máx. 10)
<code>marca</code>	string	No
<code>color</code>	string	No

Uso en app: `users-api.ts` → antes de `POST /reservas`.

6.3 Parknear Zones Service (`parknear-zones`)

Prefijo: `/zonas`

Método	Ruta	Roles
POST	<code>/zonas</code>	ADMIN
GET	<code>/zonas</code>	Público
GET	<code>/zonas/:id</code>	Público
PATCH	<code>/zonas/:id</code>	ADMIN
DELETE	<code>/zonas/:id</code>	ADMIN

Body (`CreateZonaDto` / `UpdateZonaDto`): `latitud`, `longitud`, `indicaciones`, `capacidad`, `capacidad_total`, `estado_operacion` (opcional en UI admin).

Efecto reserva: `POST /reservas` decrementa `capacidad`; cancelación/completada puede restaurarla.

6.4 Parknear Reservations Service (`parknear-reservations`)

Prefijo: `/reservas`

Método	Ruta	Roles	Descripción
POST	<code>/reservas</code>	CONDUCTOR	Crear reserva
GET	<code>/reservas/:id</code>	ADMIN, CONDUCTOR	Por ID
GET	<code>/reservas/user/:id</code>	ADMIN, CONDUCTOR	Por conductor (UUID)
GET	<code>/reservas/zona/:idZona</code>	ADMIN, CONTROLADOR	Por zona
PUT	<code>/reservas/:id/state</code>	ADMIN, CONTROLADOR	Cambiar estado
PATCH	<code>/reservas/:id/extend</code>	CONDUCTOR	Extender <code>fecha_fin</code>

POST `/reservas` — Body `CreateReservaDto`

Campo	Tipo	Requerido
<code>id_conductor</code>	UUID	Sí
<code>id_zona</code>	number (≥ 1)	Sí
<code>id_vehiculo</code>	string	Sí (placa)
<code>fecha_fin</code>	ISO 8601	Sí, posterior a ahora

Cálculo de precio (servidor):

```
precio = horas × 3.500 + 5.000
```

(`horas` = diferencia entre `fecha_real_inicio` y `fecha_fin`).

Estado inicial: `pendiente`.

PUT `/reservas/:id/state` — Body `UpdateReservaStateDto`

Campo	Tipo	Requerido
<code>estado</code>	EstadoReserva	Sí
<code>fecha_fin</code>	ISO 8601	No

Acción encargado	estado
Aceptar	activa
Rechazar	cancelada
Placa coincide	completada

Endpoint pendiente (app encargado)

Método	Ruta	Roles	Nota
GET	/reservas	ADMIN, CONTROLADOR	La app lo invoca; no está en el controller . Usar /reservas/zona/:idZona .

6.5 Parknear Payments Service (parknear-payments)

Prefijos: /pagos , /mercadopago

Método	Ruta	Roles
POST	/pagos	CONDUCTOR
GET	/pagos/:idPago	ADMIN, CONDUCTOR
GET	/pagos/usuario/:idUserario	ADMIN, CONDUCTOR
POST	/mercadopago/procesar-pago	— (fallback app)
POST	/mercadopago/notificacion	Webhook MP

POST /pagos — Body **CrearPagoDto**

```
{
  "idReserva": 123,
  "datosPago": {
    "transaction_amount": 12000,
    "token": "<card_token_mp>",
    "installments": 1,
    "payment_method_id": "master",
    "payer": { "email": "conductor@ejemplo.com" }
  }
}
```

```
}
}
```

Comportamiento servidor:

1. Lee `reserva.precio` de BD.
2. Resta pagos previos **aprobados** de la misma reserva.
3. Cobra `montoPendiente` vía Mercado Pago.
4. Persiste registro en tabla `Pago`.

Respuesta:

```
{
  "pago": { "id", "id_reserva", "monto", "estado", "metodo",
  ... },
  "mercadopago": { "id", "status", "status_detail" }
}
```

7. Servicio de soporte (propuesto)

Repo: `parknear-support-service` — estado actual: esqueleto NestJS.

UI: panel admin `/admin/solicitudes` con datos mock.

Modelos: `parknear/models/soporte.ts`.

Método	Ruta	Roles	Descripción
POST	<code>/solicitudes</code>	CONDUCTOR	Crear solicitud
GET	<code>/solicitudes</code>	ADMIN	Listar (filtros: <code>estado</code> , <code>tipo</code>)
GET	<code>/solicitudes/mias</code>	CONDUCTOR	Propias
GET	<code>/solicitudes/:id</code>	ADMIN, CONDUCTOR	Detalle
PUT	<code>/solicitudes/:id/resolver</code>	ADMIN	Aceptar/rechazar + motivo

Método	Ruta	Roles	Descripción
POST	<code>/solicitudes/:id/mensajes</code>	ADMIN, CONDUCTOR	Mensaje en hilo
GET	<code>/solicitudes/:id/mensajes</code>	ADMIN, CONDUCTOR	Listar mensajes
POST	<code>/evaluaciones</code>	CONDUCTOR	Calificación 1-5
GET	<code>/evaluaciones/resumen</code>	ADMIN	Métricas

POST `/solicitudes` : `tipo` , `titulo` , `descripcion` .

PUT `/solicitudes/:id/resolver` : `resultado` (`aceptada` `rechazada`), `motivo_admin` .

8. Flujos técnicos

8.1 Autenticación

```
sequenceDiagram
```

```
participant App
```

```
participant GW as Gateway
```

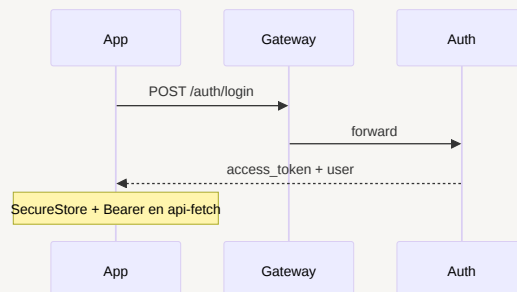
```
participant Auth
```

```
App->>GW: POST /auth/login
```

```
GW->>Auth: forward
```

```
Auth-->>App: access_token + user
```

```
Note over App: SecureStore + Bearer en api-fetch
```



8.2 Reserva y pago (conductor)

sequenceDiagram

participant App

participant GW as Gateway

participant Users

participant Res as Reservations

participant Pay as Payments

participant MP as Mercado Pago

App->>GW: POST /users/:doc/vehiculo { placa }

App->>GW: POST /reservas

Res-->>App: id, precio

App->>MP: card_tokens (public key)

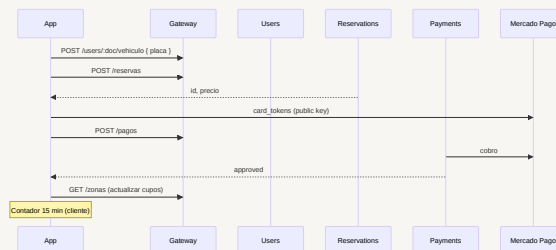
App->>GW: POST /pagos

Pay->>MP: cobro

Pay-->>App: approved

App->>GW: GET /zonas (actualizar cupos)

Note over App: Contador 15 min (cliente)



8.3 Encargado

sequenceDiagram

participant App

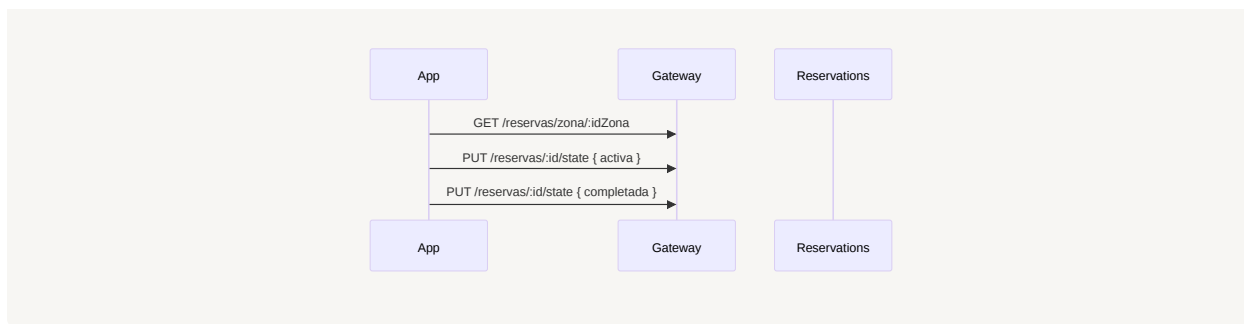
participant GW as Gateway

participant Res as Reservations

App->>GW: GET /reservas/zona/:idZona

App->>GW: PUT /reservas/:id/state { activa }

App->>GW: PUT /reservas/:id/state { completada }



9. Aplicación móvil

9.1 Stack

Tecnología	Uso
Expo SDK ~54	Runtime, build
Expo Router	Navegación (<code>app/</code>)
React Native	UI
NativeWind	Estilos
react-native-maps	Mapa
@gorhom/bottom-sheet	Detalle zona
expo-secure-store	JWT

9.2 Rutas

Ruta	Rol
<code>/</code> , <code>/login</code> , <code>/register</code>	Público
<code>/(tabs)/</code>	Conductor
<code>/admin/*</code>	Admin
<code>/encargado/*</code>	Encargado

9.3 Módulos clave

Archivo	Función
<code>auth-context.tsx</code>	Sesión, JWT
<code>mercadopago-payment-flow.ts</code>	Orquestación reserva + pago

Archivo	Función
<code>mercadopago-token.ts</code>	Tokenización tarjeta MP
<code>reservas-api.ts</code>	API reservas
<code>pagos-api.ts</code>	API pagos + fallback
<code>users-api.ts</code>	Registro vehículo
<code>zonas-api.ts</code>	Marcadores mapa
<code>active-reservation-context.tsx</code>	Timer 15 min, ruta

10. Reglas de negocio y tarifas

Concepto	Valor (COP)
Tarifa hora	3.500
Cargo reserva	5.000
Total (MP completo)	$\text{horas} \times 3.500 + 5.000$
Modo efectivo (UI)	Reserva \$5.000 vía MP; horas en zona

Regla	Dónde
Una reserva activa (UI)	<code>ActiveReservationProvider</code>
15 min para llegar	Solo cliente (no persiste en BD)
Decremento de cupo	<code>ReservasService.create</code>
Pago antes de confirmar	<code>POST /pagos</code>

11. Integración Mercado Pago

Paso	Componente
1. Token tarjeta	App → API MP (<code>EXO_PUBLIC_MERCADOPAGO_PUBLIC_KEY</code>)
2. Crear reserva	<code>POST /reservas</code>
3. Cobrar	<code>POST /pagos</code> (servidor usa <code>MERCADOPAGO_ACCESS_TOKEN</code>)
4. Webhook async	<code>POST /mercadopago/notificacion</code>
5. Fallback	<code>POST /mercadopago/procesar-pago</code> si <code>/pagos</code> retorna 500

Estados MP aprobados en app: `approved` , `accredited` .

12. Variables de entorno y despliegue

App (`parknear/.env`)

Variable	Descripción
<code>EXPO_PUBLIC_API_GATEWAY_URL</code>	URL del gateway
<code>EXPO_PUBLIC_MERCADOPAGO_PUBLIC_KEY</code>	Tokenización MP
<code>EXPO_PUBLIC_GOOGLE_MAPS_API_KEY</code>	Mapas nativos

Servicios

`JWT_SECRET` , URL de BD por servicio, `MERCADOPAGO_ACCESS_TOKEN` (payments).

Ejecución local (app)

```
cd parknear
npm install
npx expo start
```

Backend: cada microservicio NestJS + gateway en nube (`bus.parknear.online`).

13. Repositorios

Componente	URL
API Gateway	https://github.com/JacoboAlfaro/Parknear-serviceBus
Auth	https://github.com/JacoboAlfaro/parknear-auth-service
Users	https://github.com/JacoboAlfaro/parknear-users-service
Reservations	https://github.com/JacoboAlfaro/parknear-reservations-service
Payments	https://github.com/JacoboAlfaro/parknear-payments-service
Zones	https://github.com/JacoboAlfaro/parknear-zones-service
Support	https://github.com/JacoboAlfaro/parknear-support-service
App móvil	https://github.com/JacoboAlfaro/ParknearFrontend

14. Estado de implementación

Componente	Estado
Auth, Users, Zones, Reservations, Payments	✓
Mercado Pago + webhook + fallback	✓
PATCH <code>/reservas/:id/extend</code>	✓
App conductor / admin / encargado	✓
Support API	✓
GET <code>/reservas</code> (lista global)	✓
Webhook MP en producción	⚠ según despliegue

15. Referencias

- `API_ENDPOINTS.md` — contrato REST del proyecto
- Código: `parknear/lib/`, `parknear-reservations/`, `parknear-payments/`

Apéndice — Resumen de endpoints

```
AUTH:      POST /auth/register  POST /auth/login

USERS:     POST /users  GET /users  GET /users/:email
           POST /users/:documento/vehiculo  PATCH /users/:documento

ZONES:     POST /zonas  GET /zonas  GET /zonas/:id
           PATCH /zonas/:id  DELETE /zonas/:id

RES:       POST /reservas  GET /reservas/:id  GET /reservas/user/:id
           GET /reservas/zona/:idZona  PUT /reservas/:id/state
           PATCH /reservas/:id/extend

PAY:       POST /pagos  GET /pagos/:idPago  GET /pagos/usuario/:idUserario

MP:        POST /mercadopago/procesar-pago  POST /mercadopago/notificacion

SUPPORT:   POST /solicitudes  GET /solicitudes  PUT /solicitudes/:id/resolver
           POST /evaluaciones  (propuesto)
```

Manual técnico ParkNear v1.0 — documentación de arquitectura, modelos y API.